SOFTWARE REQUIREMENTS SPECIFICATION

For

MyPharma (Medicine Finder System)

Prepared by:

ROMEL IAN D. SUCANO

MARJORIE M. SURELA

KRISTEL JEAN S. UMAMBAC

Submitted in fulfillment requirements for Integrative Programming and Technologies 2 (IT305)

November 12, 2025

Table of Contents

Introduction

Finding the right medicine at the right time is crucial, especially when someone is sick and needs immediate treatment. Unfortunately, many people spend time and effort going from one pharmacy to another, only to find out that the medicine they need is not in stock. This situation causes delays in treatment, unnecessary stress, and wasted resources that could otherwise be saved.With the advancement of technology, online systems now provide a faster and more reliable way of searching for information. MyPharma is designed to help individuals find medicines online by showing the availability of specific drugs in nearby pharmacies. It also provides pharmacy details such as location and quantity, making it easier for customers to access healthcare resources without difficulty.

1.1 Purpose

This document specifies the requirements for MyPharma, a comprehensive medicine finder and pharmacy management system designed specifically for Pagadian City, Zamboanga del Sur. The system enables users to locate medicines, compare prices across pharmacies, and provides administrative tools for pharmacy management.

1.2 Intended Audience

System Administrators: Manage user accounts, pharmacy registrations, and system configuration

Pharmacy Staff: Update medicine inventory and pricing information

General Users: Search for medicines and locate nearby pharmacies

Developers: Implement and maintain the system

Stakeholders: Pharmacy owners, healthcare providers, and local government units

1.3 Product Scope

MyPharma is a web-based application that:

- Provides real-time medicine availability and pricing information
- Offers interactive mapping of pharmacy locations
- Supports multi-user roles with appropriate permissions
- Enables pharmacy management and inventory tracking

Facilitates price comparison and route planning

1.4 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification

UI: User Interface

API: Application Programming Interface

GPS: Global Positioning System

OSRM: OpenSource Routing Machine

PHP: Hypertext Preprocessor (server-side scripting language)

MySQL: Database management system

Admin: System Administrator with full access

Staff: Pharmacy staff with inventory management privileges

User: General user with search and view capabilities

2. Overall Description

2.1 User Characteristics

User Type	Technical Proficiency	Primary Tasks	Access Level
Admin	High	User management, pharmacy approval, system monitoring	Full system access
Staff	Medium	Inventory updates, price management	Pharmacy-specific access
General User	Basic	Medicine search, pharmacy location	Read-only access

2.2 Constraints

Technical: Requires XAMPP stack (Apache, MySQL, PHP)

Geographical: Focused on Pagadian City area

Network: Requires internet connection for map services

Browser: Compatible with modern web browsers

Mobile: Responsive design for mobile devices

2.3 Assumptions and Dependencies

Assumptions:

Pharmacies have stable internet connectivity

Users have basic smartphone/computer literacy

Pharmacy staff will regularly update inventory

Dependencies:

OpenStreetMap for mapping services

OSRM for routing functionality

Bootstrap for responsive UI components

Leaflet.js for interactive maps

3. Requirements Specifications

3.1 Functional Requirements

3.1.1 User Management

- System shall support user registration with invitation-only access
- System shall provide role-based access control (Admin, Staff, and Owner of the Pharmacy)
- System shall require admin approval for new user accounts
- System shall allow password-based authentication

3.1.2 Pharmacy Management

- System shall allow pharmacy registration with location data
- System shall require admin verification for pharmacy listings
- System shall store pharmacy contact information and coordinates
- System shall display pharmacy locations on interactive maps

3.1.3 Medicine Search and Inventory

- System shall provide medicine search by brand or scientific name
- System shall display medicine availability across pharmacies
- System shall show price comparisons between pharmacies
- System shall track medicine stock levels

System shall maintain price change history

3.1.4 Mapping and Navigation

- System shall display pharmacies on interactive maps
- System shall provide route planning to selected pharmacies
- System shall support GPS-based user location detection
- System shall offer both road routing and straight-line navigation

3.1.5 Administrative Functions

- System shall provide dashboard for admin oversight
- System shall allow admin approval/rejection of registrations
- System shall generate invitation tokens for new users
- System shall track system notifications and activities

3.2 Non-Functional Requirements

3.2.1 Performance

- Search results shall load within 3 seconds
- Map rendering shall complete within 5 seconds
- System shall support concurrent access by 100+ users
- Database queries shall optimize for quick response times

3.2.2 Security

- User passwords shall be encrypted using bcrypt
- Session management shall prevent unauthorized access
- SQL injection prevention through prepared statements
- Role-based access control for sensitive operations

3.2.3 Usability

- Interface shall be responsive for mobile and desktop
- Touch-friendly design for mobile users
- Intuitive navigation with clear visual hierarchy
- Accessible to users with basic technical skills

3.2.4 Reliability

- System shall maintain 99% uptime during operational hours
- Error handling for failed external service calls
- Data backup and recovery procedures

3.3 External Interface Requirements

3.3.1 User Interfaces

- **Web Interface:** Responsive design using Bootstrap CSS framework
- **Mobile Optimization:** Touch-friendly buttons and gestures
- **Map Interface:** Interactive maps using Leaflet.js
- **Admin Dashboard:** Comprehensive management interface

3.3.2 Hardware Interfaces

- **GPS:** Location services for mobile devices
- **Touch Screens:** Support for mobile touch interactions
- **Various Screen Sizes:** Responsive design from 320px to 1920px+

3.3.3 Software Interfaces

Database: MySQL for data persistence

Mapping: OpenStreetMap tiles and OSRM routing

Web Server: Apache HTTP Server

Programming: PHP for server-side logic

3.3.4 Communication Interfaces

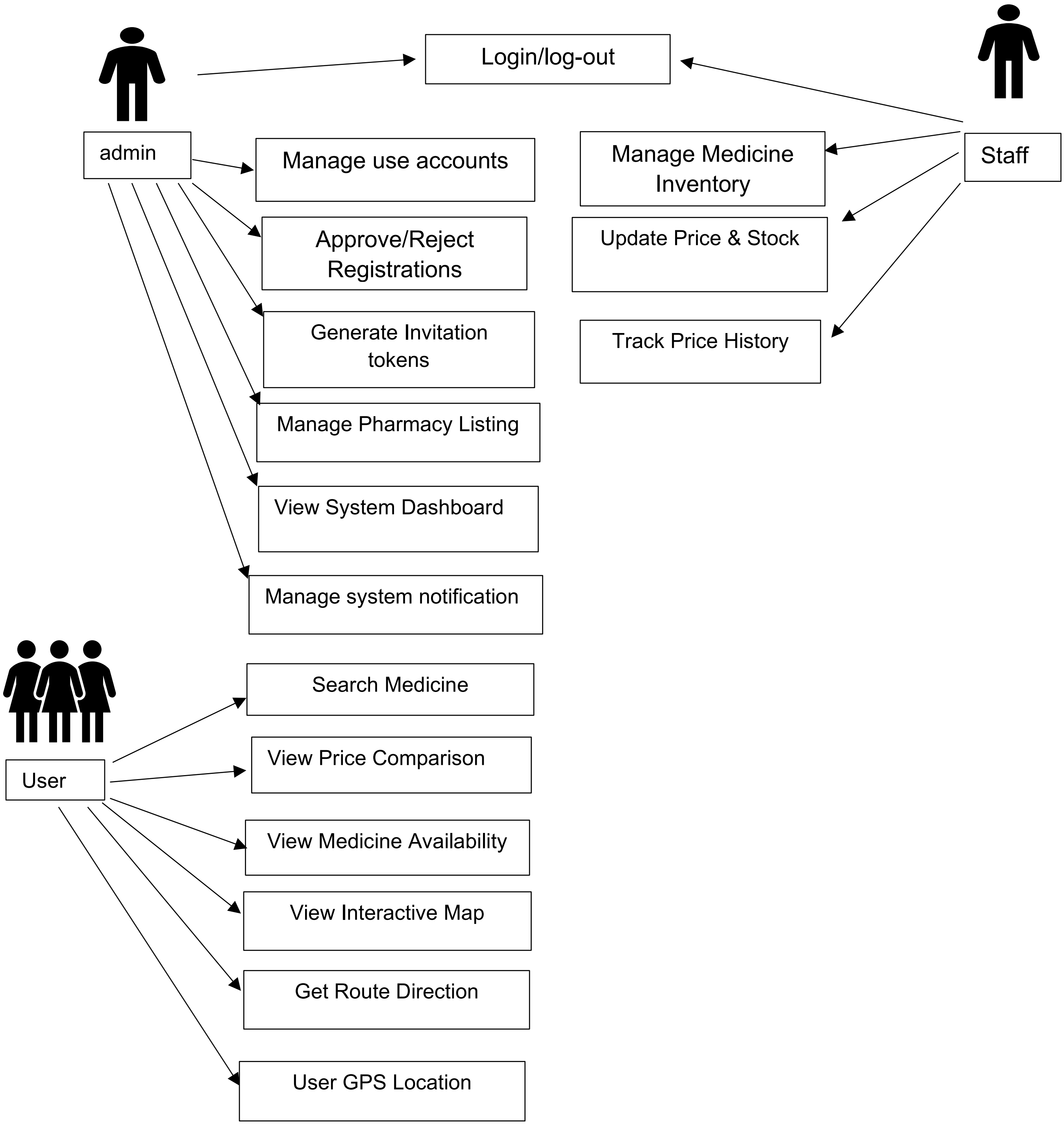
HTTP/HTTPS: Web protocol for client-server communication

RESTful APIs: JSON-based data exchange

Geolocation API: Browser-based location services

3.4 System Models

3.4.1 Use Case Diagram



3.4.2 Activity Flows

1. **Medicine Search Flow:**
    - User enters search term
    - System queries database for matches
    - Results displayed with prices and locations
    - User selects pharmacy for directions
2. **User Registration Flow:**
    - Admin generates invitation token
    - User registers using token
    - Account set to pending status
    - Admin approves or rejects account
3. **Inventory Update Flow:**
    - Staff logs into dashboard
    - Selects pharmacy and medicine
    - Updates price and stock information
    - System records change in history

3.5 Database Design

Revised Entity-Relationship Diagram

USERS Table

Structure:

- id (INT, Primary Key) - Unique identifier for each user

- name (VARCHAR(100)) - Full name of the user

- email (VARCHAR(100), Unique) - User's email address

- password (VARCHAR(255)) - Encrypted password (bcrypt)

- role (ENUM) - User role: 'pending', 'user', 'staff', 'admin'

- pharmacy_name (VARCHAR(100), Nullable) - Name of associated pharmacy (for staff)

- pharmacy_address (TEXT, Nullable) - Address of associated pharmacy

- pharmacy_lat (DECIMAL(10,8), Nullable) - Latitude coordinate of pharmacy

- pharmacy_lng (DECIMAL(11,8), Nullable) - Longitude coordinate of pharmacy

- pharmacy_contact (VARCHAR(20), Nullable) - Contact number for pharmacy

- created_at (TIMESTAMP) - Account creation timestamp

Description: Stores all user accounts in the system

PHARMACIES Table

Structure:

- id (INT, Primary Key) - Unique identifier for each pharmacy

- name (VARCHAR(100)) - Pharmacy name

- address (TEXT) - Physical address of pharmacy

- lat (DECIMAL(10,8)) - Latitude coordinate for mapping

- lng (DECIMAL(11,8)) - Longitude coordinate for mapping

- contact (VARCHAR(20)) - Pharmacy contact number

- verified (TINYINT(1)) - Approval status (0 = pending, 1 = verified)

- created_at (TIMESTAMP) - Registration timestamp

Description: Stores pharmacy information and locations

MEDICINES Table

Structure:

- id (INT, Primary Key) - Unique identifier for each medicine

- scientific_name (VARCHAR(100)) - Scientific/generic name of medicine

- brand_name (VARCHAR(100)) - Commercial brand name of medicine

Description: Master list of available medicines

PHARMACY_MEDICINES Table (Bridge Table)

Structure:

- id (INT, Primary Key) - Unique identifier for each record

- pharmacy_id (INT, Foreign Key) - References pharmacies(id)

- medicine_id (INT, Foreign Key) - References medicines(id)

- price (DECIMAL(10,2)) - Current price of the medicine

- stock (INT) - Current stock quantity available

- updated_at (TIMESTAMP) - Last update timestamp

Description: Links pharmacies to medicines with pricing and stock information

Constraints: Unique constraint on (pharmacy_id, medicine_id) combination

PRICE_HISTORY Table

Structure:

- id (INT, Primary Key) - Unique identifier for each price change record

- pharmacy_medicine_id (INT, Foreign Key) - References pharmacy_medicines(id)

- old_price (DECIMAL(10,2)) - Price before change

- new_price (DECIMAL(10,2)) - Price after change

- changed_by (INT, Foreign Key) - References users(id) - who made the change

- changed_at (TIMESTAMP) - When the change occurred

Description: Tracks historical price changes for audit purposes

NOTIFICATIONS Table

Structure:

- id (INT, Primary Key) - Unique identifier for each notification

- user_id (INT, Foreign Key, Nullable) - References users(id) - target user (NULL = all users)

- type (VARCHAR(50)) - Notification type (e.g., 'price_update', 'system')

- message (TEXT) - Notification content/message

- read (TINYINT(1)) - Read status (0 = unread, 1 = read)

- created_at (TIMESTAMP) - Notification creation time

Description: Stores system notifications for users

INVITATIONS Table

Structure:

- id (INT, Primary Key) - Unique identifier for each invitation

- token (VARCHAR(32), Unique) - Unique invitation token

- invite_type (ENUM) - Type of invitation: 'staff', 'pharmacy_owner'

- created_by (INT, Foreign Key) - References users(id) - admin who created invitation

created_at (TIMESTAMP) - Invitation creation time

used (TINYINT(1)) - Whether invitation has been used (0 = unused, 1 = used)

- used_by (INT, Foreign Key, Nullable) - References users(id) - who used the invitation

- used_at (TIMESTAMP, Nullable) - When the invitation was used

Description: Manages invitation tokens for user registration

Key Relationships

1. Users → Pharmacies: One-to-Many (Staff members can be associated with pharmacies)

2. Pharmacies ↔ Medicines: Many-to-Many through Pharmacy_Medicines bridge table

3. Pharmacy_Medicines → Price_History: One-to-Many (Each price change tracked)

4. Users → Notifications: One-to-Many (Users can have multiple notifications)

5. Users → Invitations: One-to-Many (Users can create multiple invitations)

Key Relationships:

- **Users** to **Pharmacies**: One-to-Many (Staff members belong to pharmacies)
- **Pharmacies** to **Medicines**: Many-to-Many through **Pharmacy_Medicines**
- **Pharmacy_Medicines** to **Price_History**: One-to-Many
- **Users** to **Notifications**: One-to-Many
- **Users** to **Invitations**: One-to-Many (created_by, used_by)

3.6 Implementation

3.6.1 Technology Stack

- **Frontend**: HTML5, CSS3, JavaScript, Bootstrap 5, Leaflet.js
- **Backend**: PHP 7.4+
- **Database**: MySQL 5.7+

- **Web Server**: Apache HTTP Server
- **Mapping**: OpenStreetMap, OSRM Routing

3.6.2 System Architecture

Client Layer (Web Browser)

↓

Presentation Layer (HTML/CSS/JS)

↓

Application Layer (PHP Scripts)

↓

Data Layer (MySQL Database)

↓

External Services (OSM, OSRM)

3.6.3 Security Implementation

- Password hashing using password_hash() with bcrypt
- Prepared statements for all database queries
- Session-based authentication with role validation
- Input sanitization and validation

- CORS policies for external API calls

3.6.4 Deployment Requirements

- XAMPP stack installation
- MySQL database configuration
- File system permissions
- Internet connectivity for mapping services
- Regular database backups